Page · the loop: outline, draft, probe, evidence, revise, compile, check

1 DRAFT gives the Page a promise

The DRAFT contract: what enters, what DRAFT may decide, and what it hands forward.

```
 DRAFT
+--  enters    page need . constraints . prior round
+--  owns      purpose . Aims . promised shape
+--  writes    any Page part needed to expose the design
+--  names     unknowns . dependencies . assumptions
+--  exits     one stable round contract
```

DRAFT decides what this round of the Page is trying to become.

DRAFT may create the first Page, but it may also reopen a Page that already has polished prose. It owns the Page's purpose, Aims, and promised shape for the round. A purpose says why the Page exists, and the Aims say which durable results it promises. The promised shape is the content structure needed to make those results visible.

DRAFT may add, delete, move, or rewrite material while it is deciding the promise. It may also repeat without leaving the phase while alternatives are still being tried. Its exit is not polished prose. Its exit is a stable purpose and set of Aims, plus explicit unknowns that later work can resolve.

DRAFT can place a provisional statement when its status is visible and its unknown has an owner. It cannot present unavailable evidence as settled fact. An unresolved question that matters to the promise becomes an input to PROBE.

2 PROBE resolves what the Page cannot know

The PROBE contract: a named unknown leaves the Page, meets a real source, and returns as a result.

```
 PROBE
+--  enters    question id . stake . source route
+--  owns      inquiry . retrieval . analysis
+--  writes    QA-probe . evidence record . answer
+--  returns   result . source . limits . status
+--  avoids    target Page prose
```

PROBE changes the Page's knowledge boundary without authoring the Page's argument.

PROBE starts when DRAFT, REVISE, or CHECK finds a question whose answer changes what can be supported. The question carries an id, the reason it matters, and a real route to evidence or a person. The phase ends when the answer is resolved, explicitly deferred, or reported unreachable.

PROBE records the question, source, method, result, and limits on a probe surface. It returns those records to the target Page by an explicit pointer. It does not decide how the result should be narrated or silently insert finished prose into the target Page. That landing decision belongs to REVISE.

A Page with no consequential unknown can move from DRAFT directly to CHECK. Several probes may branch from one round, and a later phase may open another probe. Skipping PROBE is valid only when no unresolved question is being hidden.

The target Page is the Page whose lifecycle raised the stake-bearing Q-consumer. A family may file the QA-probe under an evidence route such as Literature or Value without transferring the Q-consumer, A-consumer, or State to that route's topic Page. The Paper family's S03 and S04 layout exposes the distinction: a Results Page can raise the question, a QA-probe can live beneath a Value topic, and a QA-bank file can answer it. Treating physical placement as an ownership transfer gives one exchange two Page-facing consumer surfaces before the answer reaches prose.

One PROBE run may write its declared Probe surface and the active target Page's Probe reference, A-consumer, and State. When the same Q-executor serves other Q-consumers, the Probe surface records their references and makes the answer available without authoring those sibling Pages in the current run. Each sibling interprets the answer and changes its own Content through its own PROBE or REVISE route, with its own version and CHECK. This preserves one Q-executor for reuse without turning one answer into an unbounded cross-Page edit.

The active Page may need to show the Q-executor, bank target, state, returned answer, and limits while keeping its source concise. A read-only projection can render that chain from the QA-probe and bank answer, while the Page source keeps the Probe reference and its own A-consumer interpretation. The same reference should drive phase-scoped context loading and CHECK, so an author does not maintain a Probe pointer, an embed, and a Related Board Pages row for one relationship. Missing targets, superseded answers, or changed source hashes must fail visibly rather than leave an old projection looking current.

3 REVISE makes the current promise work

The REVISE contract: the Page changes while its purpose and Aims remain fixed.

```
 REVISE
+--  enters    current Page . evidence . feedback
+--  holds     purpose . Aims
+--  owns      add . delete . move . rewrite
+--  updates   Content . Opening . States . records
+--  exits     stronger version . visible remaining gaps
```

REVISE improves the realization of the current round rather than redefining its promise.

REVISE uses landed evidence, feedback, and close reading to make the current Aims true. It may add a paragraph, delete a section, move an argument, or rewrite the Opening. Large edits are still REVISE when the same purpose and Aims continue to describe the result.

If the purpose and Aims remain accurate without amendment, the work is REVISE. If the work changes what the Page is for or what it promises to establish, the work returns to DRAFT. The amount of changed text and the age of the Page do not decide the phase.

When revision exposes a consequential unknown, REVISE names it and routes it to PROBE. Evidence that landed is interpreted and woven into the Page. Evidence that did not land remains a visible limit, a deferred pointer, or a reason the round cannot close.

4 CHECK decides where the current version goes next

The CHECK contract: one concrete version is judged against its promise and routed.

```
 CHECK
+--  enters    rendered version . Aims . evidence . constraints
+--  owns      judgment . findings . closing decision
+--  writes    comments . findings . gate record
+--  routes    close . REVISE . PROBE . new DRAFT . HOLD
+--  avoids    curing its own substantive finding
```

CHECK observes and decides; another phase performs the content change it requests.

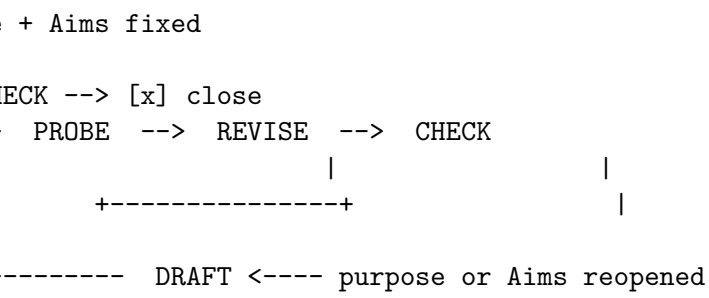
CHECK reads the rendered Page against its Aims, evidence, inherited constraints, and closing rule. Mechanical checks may run throughout the work, but the CHECK phase is the gate that decides the version's fate. The kind of Page supplies the gate, such as a self-audit, human approval, or an external shipping event.

A passing version closes the round. A content or clarity problem routes to REVISE. A missing answer routes to PROBE and then normally to REVISE. A broken purpose or Aim routes to a new DRAFT round. Missing authority, evidence, tools, or a pending human ruling routes to HOLD, the honest stop that §9 gives the automatic loop.

CHECK may add comments, findings, and a gate record. It does not quietly rewrite the content and call the same version checked. If the checker also performs the fix, the work changes phase explicitly and the resulting version is checked again.

5 Transitions form rounds, not a rigid conveyor belt

The transition grammar: repetition is legal, and returning to DRAFT changes the round.



The arrows express dependencies and routing choices, not a rule that every phase runs once.

`DRAFT, DRAFT, PROBE, REVISE, DRAFT` is a legal history. The first two DRAFT entries can be repeated design work in one round. The final DRAFT means that revision reopened the promise and therefore began a new round. A complete draft with no unknowns and no revision need may go directly to CHECK.

REVISE may reveal that the current purpose or Aims are wrong. The Page then returns to DRAFT and receives a new round, while the Page itself persists. Earlier evidence remains available, but its relevance must be checked against the new promise. Any earlier closing decision no longer applies to the new round.

The history should say which authority was used: promise reopened, unknown resolved, current promise improved, or version judged. Without that reason, two identical diffs can be mislabeled as different phases and two different intentions can be mislabeled as the same one.

6 Operations route by reason, not by edit shape

The same operation under two authorities: a compact test for ordinary Page changes.

operation	DRAFT when	REVISE when
add	new promise or Aim	serves a fixed Aim
delete	removes a promise or Aim	removes weak or extra prose
move	changes the promised argument	improves flow under the promise
rewrite	reframes purpose or Aim	improves supported expression

The operation says what changed in the file; the authority says which phase performed it.

Adding a paragraph to explain evidence for an existing Aim is REVISE. Adding a paragraph because the Page now answers a new question is DRAFT. Deleting unsupported or duplicate prose while keeping the Aim is REVISE. Deleting the promised result itself, or deleting its Aim, is DRAFT.

Moving paragraphs to improve flow under the same argument is REVISE. Changing the argument the Page promises to make is DRAFT even if the diff is only one moved heading. Rewriting the Opening for clarity is REVISE when the purpose stays fixed. Rewriting it to give the Page a different purpose is DRAFT.

PROBE writes questions, sources, evidence, and answer records. CHECK writes findings, comments, and gate records. When either phase causes target prose to change, the actual content edit is performed under REVISE or a restarted DRAFT.

7 One lifecycle Page holds the phases together

The Page split rule: each phase begins as a Content division and earns a Page only by becoming an independent question.

```
 QB5 . ONE LIFECYCLE QUESTION
+--  1 .  DRAFT
+--  2 .  PROBE
+--  3 .  REVISE
+--  4 .  CHECK
+--  5 .  transitions
+--  6 .  operations

 split test . ALL required
+--  independent question
+--  own Aims +  States
+--  independent closing gate
+--  own continuation files

 phase skill   executable contract
 design Page   independently closable question
 mapping       no automatic 1:1 mirror
```

Shared boundaries stay together until one phase can carry and close a question of its own.

DRAFT, PROBE, REVISE, and CHECK remain divisions of QB5 because their meanings depend on comparison and transition. The DRAFT and REVISE boundary is easier to judge when both definitions and the operation examples stay on one Page. The transition rules also need one home that no phase-specific Page can own alone.

The phase must have an unresolved question that is not merely asking for its definition. It must need its own Aims and States rather than borrowing QB5's records. It must have an independent closing gate and its own small continuation map in Files. If any test fails, the material remains a division or paragraph on QB5.

A phase skill may remain separate because a worker needs to load one execution contract at a time. That file boundary does not force the Board to create a matching design Page. If a phase later passes the split test, QB5 keeps the shared boundaries and transitions while pointing to the new Page for that phase's independent question.

8 Page Type and Page Phase are separate skill axes

The skill composition: the base resolves what the Page is and how the current work is acting on it.

```
 haipipe-page                    the shared Page contract and router
+--  page-types/                      what kind of Page persists . ten types . QB6 owns the ros
|   +-- for-stage
|   +-- for-skill
|   +-- for-venue
|   +-- ... seven more . for-design . for-display . for-literature . for-meeting . for-section
+--  page-phases/                     what authority acts now
    +-- draft
    +-- probe
    +-- revise
    +-- check

one invocation = base + matching Page Type + current Page Phase + family worker
```

Page Type and Page Phase are orthogonal, so their folder names and skill names must not collapse them into one label.

`haipipe-page-for-stage`, `-for-skill`, and `-for-venue` keep their names and move under `page-types/`. The roster has since grown to ten Page Types, and `QB6` owns the admission test and the list. The grouping folder is organizational and carries no `SKILL.md` of its own. A new Page Type is added only when a persistent Page needs a structural contract that the base does not provide.

The phase contracts are `haipipe-page-draft`, `-probe`, `-revise`, and `-check` under `page-phases/`. They apply across Page Types and therefore do not use `for-stage` in their names. The base first adopted the phase vocabulary without adding `ADVANCE`. The automatic router now earns a verb named `RUN`, because it may repeat, branch, HOLD, or return to DRAFT rather than advance in one direction.

The target Page owns the stake-bearing Q-consumer. The PROBE phase strips the stake into a neutral Q-executor, binds the returned A-executor, and writes an A-consumer interpretation for each consumer. One Q-executor may serve several Q-consumers. `haipipe-probe` remains the shared crossing protocol, while `haipipe-page-probe` applies that protocol to a Board Page.

9 RUN turns the phase grammar into a bounded loop

The executable flow: one controller composes separate producer, builder, and judge roles without prescribing one phase sequence.

```
 raw-material packet
|  Page . Type . start Phase . intent . sources . constraints . gate . limits

 controller
+--  DRAFT /  PROBE /  REVISE  -> producer
+--  build + version snapshot       -> mechanical builder
+--  CHECK exact version            -> fresh read-only judge
                  |
                  +-- [x] CLOSE
                  +--  REVISE ------+
                  +--  PROBE -------+
                  +--  DRAFT round+1|
                  +--  HOLD          |
                                      +--> controller
```

RUN follows returned authority routes and stops at explicit terminals or limits. Figure 1 states the same controller as an algorithm, so a reader can see the three actors alternate and read the stop conditions in the order the loop tests them.

packet : $\Pi = (\mathit{page}, \mathit{type}, \varphi_0, \mathit{intent}, \mathit{sources}, \mathit{constraints}, \mathit{gate}, \mathit{limits})$
receipts: $\rho = (r_1, \ldots, r_n)$, one receipt per attempted phase

$\varphi \leftarrow \varphi_0$ `// an existing Page enters at CHECK, a new one at DRAFT`
$s \leftarrow 0; \quad k \leftarrow 1$ `// steps taken, and the current round`
while $s < \mathit{limits.max\_steps}$ **and** $k \leq \mathit{limits.max\_rounds}$ **do**
  $\Gamma \leftarrow \textsc{Context}(\mathit{page}, \varphi)$ `// one hop, this phase's rows only; a bad row is a HOLD`
  if $\varphi \in \{\textsc{Draft}, \textsc{Probe}, \textsc{Revise}\}$ **then**
    $(v', \hat{r}) \leftarrow$ `Producer`(φ, Π, Γ) `// one phase, by one actor, who also proposes a route`
    $v' \leftarrow$ `Builder`(v') `// rebuild, deterministic checks, version identity`
  else
    $(v', \hat{r}) \leftarrow$ `Judge`(v, Π, Γ) `// a fresh read-only actor, judging one exact version`
  $r \leftarrow \textsc{Receipt}(\varphi, \mathit{actor}, v \to v', \hat{r})$; append r to ρ
  if $\hat{r}.\mathit{route} \notin R(\varphi)$ **then return** ρ ***with*** HOLD
  if $\hat{r}.\mathit{route} \in \{\textsc{Close}, \textsc{Hold}\}$ **then return** ρ
  if $\hat{r}.\mathit{route} = \textsc{Draft}$ **and** $\hat{r}.\mathit{reopens}$ **then** $k \leftarrow k + 1$
  $\varphi \leftarrow \hat{r}.\mathit{route}; \quad v \leftarrow v'; \quad s \leftarrow s + 1$
return ρ ***with*** LIMITREACHED `// did not converge, which never means the Page passed`

Figure 1: RUN, as the router it is rather than the conveyor belt its name could suggest. The producer, the builder, and the judge are three separate actors; the loop follows the route each returns, refuses a route outside $R(\varphi)$, and stops at CLOSE, HOLD, or a limit.

ADVANCE suggests that one phase has one next phase and that progress always moves forward. RUN means execute the current Page lifecycle from a named starting authority until CLOSE or HOLD. The legal route table is dynamic: DRAFT, PROBE, and REVISE may repeat or hand off, while only CHECK may CLOSE. Figure 2 writes that table as two rules and derives both laws from them, including one this prose never states: CHECK cannot route to CHECK, so a judged version reaches a second judgment only through a producer. Returning to DRAFT from another phase increments the round only when purpose or an Aim reopened.

$$
\begin{aligned}
\Phi &= \{\textsc{Draft},\ \textsc{Probe},\ \textsc{Revise},\ \textsc{Check}\} && \text{the four phases}\\
T &= \{\textsc{Close},\ \textsc{Hold}\} && \text{the two terminals}\\
R(\varphi) &= \Phi \cup \{\textsc{Hold}\} && \text{for } \varphi \in \Phi \setminus \{\textsc{Check}\}\\
R(\textsc{Check}) &= \big(\Phi \setminus \{\textsc{Check}\}\big) \cup T && \text{the judge's own row}
\end{aligned}
$$

Two consequences follow from the second pair of lines alone:

$$
\begin{aligned}
\textsc{Close} \in R(\varphi) \iff \varphi = \textsc{Check} && \text{only a judge may close}\\
\textsc{Check} \notin R(\textsc{Check}) && \text{no version is judged twice untouched}
\end{aligned}
$$

Figure 2: The legal-route relation R, and the two laws it makes checkable. A producer may hand off to any phase including itself; the judge alone reaches CLOSE, and alone cannot reach itself, so every judged version passes through a producer before it is judged again.

The packet names the persistent Page, stable Page Type, starting Phase, run intent, source paths, settled constraints, closing gate, and step and round limits. A new Page is first created and registered, then RUN starts it at DRAFT. An existing Page with no known next need starts at CHECK, allowing a cold judge to route the visible version. A missing source, unknown gate, or ambiguous authority becomes HOLD rather than an invented input.

The producer performs exactly one DRAFT, PROBE, or REVISE phase and suggests a legal route. The mechanical builder rebuilds, runs deterministic checks, and identifies the source plus render version. The fresh reviewer performs CHECK against that exact version and returns CLOSE, REVISE, PROBE, DRAFT, or HOLD. The controller validates and follows the route, but it cannot alter Page prose or convert a pending human gate into CLOSE.

The run stops on CLOSE, explicit HOLD, missing input, failed build, version mismatch, required human ruling, maximum steps, or maximum rounds. Reaching a limit says the process did not converge within its budget. It never says the Page passed. The run's final state and every attempted phase remain inspectable instead of disappearing into an agent transcript.

10 Audit proves process claims with receipts and fault tests

The assurance model: deterministic invariants, fresh semantic judgment, and direct or human evidence support different quality claims.

```
phase receipts             what happened, by whom, to which version
deterministic audit        legal routes . rounds . roles . versions . bounds
fresh CHECK                function . evidence . readability . local gate
human evidence             approval only where the Page Type requires it
branch + fault tests       expected success and expected refusal
```

A passing process audit proves that the declared process ran correctly, not that every possible substantive claim is true.

Each receipt records step, round, Phase, producer or judge actor, builder actor, status, source and render SHA-256 values, route, reason, artifacts, evidence, findings, and human-gate state.

CHECK additionally binds `checked_version` and a verdict. The receipts are ordered and stored outside Page discovery under `_runs/page/<page-id>/<run-id>.json`. The deterministic auditor independently rehashes the source and rendered files on disk; matching claims repeated inside a receipt are not accepted as artifact evidence by themselves. The terminal CHECK record is not appended to the Page afterward, because that append would change the version it claims to approve.

The preserved raw-material packet must match the run identity, first Phase, declared gate, and limits. Only legal phase routes are accepted, and only CHECK may CLOSE. A non-DRAFT route to DRAFT must name the reopened purpose or Aim and increment the next round exactly once. The producer, mechanical builder, and judge of one version must have different actor identities. Every version id must be the two declared lowercase SHA-256 digests joined by `:`, and every receipt must begin from the preceding receipt's ending version. CHECK must observe identical before, after, and checked ids; the auditor rehashes the current artifacts, and any later change requires a new CHECK. A required human gate closes only with durable evidence that the person ruled. Maximum steps and rounds terminate as non-convergence, never as a pass.

Happy paths include DRAFT directly to CHECK and DRAFT through optional PROBE and REVISE. Branch tests include CHECK to REVISE and back, CHECK to PROBE, and CHECK to a new DRAFT round. Fault injection includes self-approval, mutation after CHECK, illegal route, packet mismatch, broken version continuity, symbolic hashes, missing human evidence, failed worker, non-terminal trace, and exhausted limits. The deterministic harness must reject each injected fault for the specific invariant it violates.

The mechanical checker can prove structural facts, and the lifecycle auditor can prove process facts. A fresh reviewer supplies semantic evidence that the Page performs its declared function and is readable without the drafting conversation. Direct sources or a human gate supply claims those instruments cannot settle. The final report therefore names the checked version, traversed branches, evidence inspected, remaining findings, gate state, and residual risk instead of saying only that quality is guaranteed.